When Does the Dependency Graph Help Code-to-Architecture Mapping?

Rakhshanda Jabeen[a,c,*], Morgan Ericsson[a], Jonas Nordqvist[b] and Anna Wingkvist[a]

[a]Department of Computer Science, Linnaeus University, Växjö, Sweden
[b]Department of Mathematics, Linnaeus University, Växjö, Sweden
[c]Electrolux Professional AB, Ljungby, Sweden

ARTICLE INFO

Keywords:
Software Architecture
Code-to-Architecture Mapping
Graph Neural Networks
Semi-Supervised Learning

ABSTRACT

Architectural technical debt arises when a system's implementation drifts from its intended modular decomposition, while the mapping between source files and architectural modules is missing, incomplete, or costly to maintain manually. We formulate code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, with files as nodes, typed dependencies as edges, and lexical features derived from code identifiers and directory layout. Starting from a small labeled sample, an iterative self-training procedure expands the mapping to the remaining files. We compare a graph convolutional network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP), using identical features and training procedures for the learned models, against a Naive Bayes reference. Across 100 runs on ten open-source Java and C/C++ systems, all three neural models outperform the reference on both F1 metrics. GAT achieves the highest averages of system-level median macro and micro F1, at 83.5 and 92.2, respectively, whereas MLP remains competitive or superior on several systems. On Chrome, the largest system with 18,343 files, GAT and GCN exceed MLP by 22 and 19 macro-F1 points, respectively, and have substantially fewer persistently misclassified files. Connectivity and homophily are associated with where neural message passing helps, while multi-relational aggregation outperforms a single merged edge type on every system for both GCN and GAT. A follow-up study finds that several model rankings depend on the training seed, qualifying smaller advantages without removing the large graph benefit observed on Chrome.

1. Introduction

Software architecture defines a system's high-level structure and the dependencies among its principal elements, and it provides the basis for reasoning about maintainability, security, and other quality attributes [7]. That reasoning requires the implementation to stay aligned with the intended design. When it does not, the gap is architectural technical debt [21]: cheap to introduce, but diffuse and relational rather than local to a single file, and therefore expensive to detect and repay once it accumulates. Large language models (LLMs) are increasingly used to generate or assist with code commits [10], motivating renewed attention to whether local code changes remain consistent with a system's module structure. An empirical study of commercial AI code generators found clones between generated functions and existing open-source code [38]. These observations motivate conformance checking, but do not establish that LLM-generated code causes architectural erosion. Architectural erosion more generally concerns drift between the intended design and its implementation [9].

Detecting this drift requires comparing the intended architecture against the implemented one. Static Architecture Conformance Checking (SACC) addresses this comparison, and reflexion modeling is a SACC technique that classifies relations between the intended and implemented architectures as convergent, divergent, or absent [26, 27]. However, reflexion modeling depends on a mapping between source code entities and architectural modules, and such mappings are often missing, incomplete, or expensive to maintain manually [1]. Even for medium-sized systems, recovering or maintaining the architectural decomposition can require substantial expert effort [13]. Automated code-to-architecture (C2A) mapping is therefore an important prerequisite for practical conformance checking.

Prior work on C2A mapping has explored supervised and recommendation-based approaches. Supervised approaches compute *attraction values* that quantify the association strength between source code entities and architectural modules [6, 3]. Olsson et al. [29] proposed a semi-supervised Naive Bayes mapper that combines syntactic and semantic information to assign source code entities to architectural modules. InMap [33] took a recommendation-based view instead, using information retrieval over the codebase to suggest candidate mappings with minimal architectural documentation and no training data.

A separate line of work targets architecture recovery, where the decomposition itself must be discovered rather than assumed. Recent recovery approaches include graph neural networks (GNNs) combined with contrastive learning and autoencoders [15, 17], along with language models applied directly to source code for recovering design

*Corresponding author
rakhshanda.jabeen@electroluxprofessional.com (R. Jabeen); morgan.ericsson@lnu.se (M. Ericsson); jonas.nordqvist@lnu.se (J. Nordqvist); anna.wingkvist@lnu.se (A. Wingkvist)
ORCID(s): 0009-0002-0430-8179 (R. Jabeen); 0000-0003-1173-5187 (M. Ericsson); 0000-0002-0510-6782 (J. Nordqvist); 0000-0002-0835-823X (A. Wingkvist)

structures [2], for agent-based recovery of large legacy systems [30], and for extracting architectural entities to support traceability [12]. The distinction from C2A mapping matters here: the module set is unknown in recovery, whereas in mapping it is given by the intended architecture, and the task is to assign each file to one of those modules. It also matters for evaluation, since many recovery benchmarks are open source, and a decomposition reproduced by a pretrained model may reflect code understanding, prior exposure to the system, or both. Mapping methods that learn from a system's own structure and can be evaluated against a known ground truth therefore remain important.

In this research, we study C2A mapping[1] as semi-supervised node classification over a multi-relational directed dependency graph. Source files are nodes, typed static dependencies between them are edges, and node features encode code identifiers via Word2Vec embeddings together with directory context. A small set of labeled files is expanded into a complete mapping through iterative self-training. We compare three encoders under the same features and training procedure: a graph convolution network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP). Holding the feature representation, the classifier head, and the self-training loop fixed is intended to isolate what the dependency graph contributes beyond the node features alone, though the encoders themselves differ in architecture beyond the presence or absence of message passing.

We evaluate the approach on ten open-source systems written in Java and C/C++, reporting mapping accuracy and its variation across runs. This paper extends our earlier study [14] in five respects: it broadens the benchmark by adding C/C++ systems; replaces the earlier bag-of-words representation of code identifiers with Word2Vec embeddings; adds GAT to assess the effect of attention-based aggregation on the mapping task; introduces an MLP baseline to assess the contribution of dependency information; and corrects a label-assignment defect in the earlier self-training pipeline. We further evaluate the corrected self-training procedure, conduct new ablation studies of model depth and edge-aggregation direction, and examine the sensitivity of selected comparisons to the training seed.
Our study is guided by the following research questions:

- **RQ1.** How effectively does GNN-based C2A mapping perform relative to non-graph baselines, and how stable is its performance across runs?
- **RQ2.** What does the dependency graph contribute beyond textual features under an identical training procedure, and which system properties are associated with where it helps?
- **RQ3.** How sensitive is the approach to modeling dependency types separately, aggregation depth, and edge direction?

[1] Source code is available at https://github.com/sse-lnu/GCN-SAEroCon25.

2. Related Work

Tzerpos and Holt [35] introduced the orphan adoption problem, in which source code entities that are not yet part of the documented architecture are incrementally assigned to modules. Christl et al. [6] built on this idea with the Human-Guided Clustering Method (HuGMe), a semi-automatic approach that recommends mappings through attraction functions and leaves the final decision to the architect. Bittencourt et al. [3] studied attraction functions derived from information retrieval, where an entity is assigned to the module whose textual content it most resembles. These approaches rely on two signals that persist in later work: lexical similarity between an entity and a module and dependency proximity to entities that are already mapped, but they combine them through fixed, manually specified rules.

Subsequent work replaced these rules with learned models. Olsson et al. [29] proposed a semi-supervised Naive Bayes mapper that combines lexical information from source code with dependency-derived information through Concrete Dependency Abstraction, and reported that 15–20% of the entities must be mapped beforehand to reach acceptable performance. Florean et al. [11] compared text classifiers for the same task and showed that both the classifier and the text representation affect mapping quality. InMap [33] instead ranks candidate mappings interactively, using natural language module descriptions together with source code, which removes the need for a pre-mapped training set. In all of these, dependencies enter the model only after being flattened into features attached to individual entities, so the relational structure of the system is consumed rather than represented.

Language models have recently been applied to the same task. Rukmono et al. [32] use chain-of-thought prompting with GPT-4 to deductively assign implementation units to a predefined reference architecture. Johansson et al. [19] evaluate language model embeddings alongside TF-IDF and GloVe as input to method-level classifiers. Follow-up work links variation across projects most strongly to class separability and density in the embedding space [18]. A parallel line applies language models to architecture recovery, where the module set is inferred rather than given [2, 30]. Pretrained models capture semantic cues that engineered text features miss. ArchAgent, for example, also incorporates reference graphs and related-file context when summarizing code. Our approach instead models dependency types directly through relation-specific message passing.

Graph representation learning uses dependencies as structure rather than as features. NEGAR [5] showed that random-walk embeddings over the dependency graph, followed by clustering, recover meaningful architectural structure. GAER [15] extended this to heterogeneous graphs, comparing GCN and GAT encoders trained within a graph autoencoder and modeling each dependency type as a separate relation, and [17] reports related results using contrastive objectives. Both are unsupervised recovery methods, so the module set is discovered by clustering rather than fixed in advance as it is in mapping. Our earlier work [14] took this

step and evaluated GCN-based models on homogeneous and dependency-type-aware graphs, showing that a small labeled set can be expanded through self-training. That study was limited to Java systems of moderate size, used a single graph-based encoder family, and did not isolate how much of the observed performance came from the dependency graph as opposed to the node features.

The present work addresses these limitations. We evaluate GNN encoders together with a graph-free neural baseline that receives identical features and an identical self-training loop, to examine the contribution of dependency information under a common input representation and self-training schedule. Differences in encoder depth, dropout, and parameter count remain potential confounders.

3. Methodology

This section presents our approach to C2A mapping as semi-supervised node classification over a directed multi-relational dependency graph. We first formulate the mapping task, then describe the graph and feature construction, introduce the encoders compared in this study, and finally the self-training procedure used to grow an initial labeling into a full mapping. Figure 1 illustrates the heterogeneous GNN encoder at the core of this pipeline.

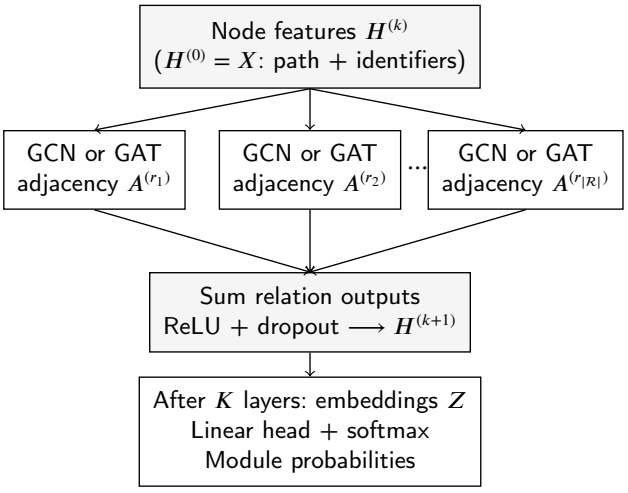


Figure 1: Overview of the heterogeneous GNN encoder. Node features X (file-path indicator + Word2Vec identifiers) and relation-specific adjacency matrices $A_{\mathcal{R}}$ are passed through a relation-specific encoder (GCN or GAT), whose outputs are summed and passed through a non-linear activation function (ReLU) to produce embeddings $Z \in \mathbb{R}^{|\mathcal{F}| \times d_h}$, where d_h is the number of hidden channels. A linear layer maps Z to per-module logits, and a final softmax produces the class probabilities used for classification and pseudo-labeling.

3.1. Problem Formulation

Given a software system with a set of files $\mathcal{F}$ and a fixed set of architectural modules $\mathcal{C}$, a C2A mapping assigns each file $i \in \mathcal{F}$ to a module $y_i \in \mathcal{C}$. A small subset $L \subset \mathcal{F}$ is labeled with its ground-truth module; the remaining files $U = \mathcal{F} \setminus L$ are unlabeled, and the task is to recover the assignment for every file in U. We consider a transductive setting in which the lexical node features and dependency structure of all files are available during training, while module labels are available only for files in L.

Two sources of information are available for each file. The first is intrinsic and consists of what the file itself declares (identifiers) and where it sits in the source tree (directory structure), and it is available for every file regardless of how the file is connected to the rest of the system. The second is relational: the representations of its neighbors in the dependency graph, propagated through message passing and available only where the file has neighbors. We frame the task as representation learning so the two sources can be treated separately: every encoder maps a file i to an embedding z_i, and a shared classification head maps z_i to a module, so the encoders differ in what evidence enters the encoder to generate z_i and not in how it is used.

3.2. Graph Construction

We represent a software system as a heterogeneous graph $G = (X, A_{\mathcal{R}})$ over its files, following the construction introduced for unsupervised recovery in GAER [15]. Code identifiers and directory paths are file-level properties and are encoded in the feature matrix X. Static dependencies are file-to-file relations, each belonging to a dependency type r drawn from the set of relation types $\mathcal{R}$, and encoded as edges in the multi-relational adjacency $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$.

3.2.1. Node Features

The intrinsic evidence for a file comes from two conventions developers follow when they write and organize code. Identifiers name the concepts a file works with, so class, method, and variable names carry the vocabulary of the domain the file serves and, with it, an indication of the responsibility it implements [40]. Directory paths record how developers chose to group files, and that grouping frequently tracks the subsystem decomposition the architecture describes [13]. The two are complementary where structure alone is ambiguous, since a widely used utility file may depend on code throughout the system while its name and its location still indicate what it is for. Both signals are available after identifier extraction, without requiring additional dependency resolution during feature construction.

We normalize code identifiers by splitting on *camelCase*, *PascalCase*, and *snake_case* boundaries and removing low-information tokens (stopwords, very short tokens, tokens appearing in over 90% of files), then train a 128-dimensional skip-gram Word2Vec [25] embedding over the remaining tokens for 20 epochs with a context window of 5, a minimum token count of 5, and a vocabulary-pruning limit of 2,000 during vocabulary construction. We use one document per file, a fixed stopword list, and a single Word2Vec worker with a fixed seed. Let T_i denote the resulting sequence of normalized tokens for file i; its textual representation is the ℓ_2-normalized average of the Word2Vec embeddings of the tokens in T_i. File paths are tokenized separately and represented as a binary bag-of-words vector. Common

leading directories and the generic components `src`, `main`, and `java` are removed. Java files use their remaining directory tokens, falling back to the file name when none remain; C/C++ files also include base-name tokens. Tokens are lowercased and vectorized using the default word-token pattern of `CountVectorizer`. For file i, the two are concatenated:

$$x_i = \left[\text{Binary}(\text{FilePath})_i \,\middle\|\, \text{Word2Vec}(T_i)\right]. \tag{1}$$

Stacking x_i over all files gives the feature matrix X. Every encoder in this study receives the same X, so the intrinsic evidence available to each of them is identical by construction, although the encoders also differ in depth and parameter count.

3.2.2. Static Dependencies (Edges)

Files that implement related functionality tend to depend on one another directly or through short dependency paths, which is what makes the dependency graph informative for architecture recovery [16]. Dependency types are not interchangeable in this respect, and call edges have been argued to indicate tighter coupling than type-usage edges [5, 34]. Collapsing them into one adjacency matrix removes the distinction between dependency types, so we keep one adjacency matrix per type and let the encoder learn a separate transformation for each. Section 5 evaluates this relation-typed design against a homogeneous graph for both GNN encoders.

We extract dependencies with the multilingual static-analysis tool *Depends* [39], which resolves the typed relations listed in Table 1 between code entities across our Java and C/C++ systems. Because the mapping task operates at file granularity, member-level dependencies such as a call between two methods are aggregated into edges between their enclosing files, and repeated dependencies of the same type between the same ordered pair collapse into a single edge, so $A^{(r)}$ records whether a relation holds and not how often it occurs.

Table 1
Static dependencies used in the software graph.

Dependency	Description
`Annotation`	Annotation relation to an annotated entity.
`Call`	Function or method invocation.
`Cast`	Conversion of an entity to another type.
`Contain`	Definition of a variable or field of another type.
`Create`	Creation of an instance or program entity.
`Extend`	Inheritance or parent–child type relation.
`Implement`	A class implements an interface or abstraction.
`Import`	A file imports other classes, methods, or variables.
`Parameter`	Use of a type as a method or function parameter.
`Return`	Use of a type as a return type.
`Throw`	A type is used as an exception.
`Use`	Use or assignment of variables.

`Annotation` and `Throw` are not observed in the C/C++ systems.

A static dependency is asymmetric, and $A^{(r)}$ preserves that direction: an edge points from the file where it originates to the file it refers to. For file i under relation r, we distinguish its in-neighborhood, the files that depend on it, from its out-neighborhood, the files it depends on. A file with a large in-neighborhood and a small out-neighborhood provides functionality, the profile of an interface or utility, and its clients may be spread across many modules. A file with the opposite profile consumes functionality, like orchestration or entry-point code, and what it draws on says more about what it coordinates than about which module it belongs to. Which of the two an encoder aggregates over, or whether it discards the direction altogether and treats the edge as undirected, is therefore a modeling choice rather than a fixed property of the graph; Section 3.3 defines it as a parameter of message passing, and Section 5 tests its impact on mapping.

3.3. Representation Learning and Mapping

We compare three encoders that operate on the same node features X. The graph-based models, GCN and GAT, use the relation-specific adjacency $A_{\mathcal{R}}$ to aggregate information from neighboring files, while the non-graph baseline uses X alone. All models take $h_i^{(0)} = x_i$ as input.

The GNNs stack K message-passing layers to produce $z_i = h_i^{(K)}$, while the MLP uses one hidden layer. Each encoder feeds its embedding into a linear classification head followed by softmax to obtain module probabilities. The encoders differ in their use of neighbors, aggregation weights, depth, and parameter count.

3.3.1. GNN Encoders

Both GNN encoders update the representation of each file by aggregating messages from its neighbors under each dependency type r, a mechanism known as message passing. The neighbors of file i under relation r form the set $\mathcal{N}_i^{r,\pi}$, where the propagation scheme π determines which side of the directed edge is aggregated over: forward, reverse, or undirected, as introduced in Section 3.2.2. A separate relation- and layer-specific kernel $W^{r,(k)}$ transforms each neighbor's message, so a call edge and an inheritance edge contribute through different learned transformations rather than being folded into a single adjacency. Each additional layer widens this neighborhood by one hop, so a K-layer encoder lets information reach a file from up to K dependency steps away. We add self-loops before aggregation so a file's own features are included in its update, which matters when its neighborhood is empty. The neighborhood notation in Equations 2 and 3 includes the self-loop. Forward propagation is source-to-target: a file aggregates from files that depend on it. Reverse propagation aggregates from files it depends on, and undirected propagation uses both, with duplicate edges removed per relation.

GCN Following Kipf and Welling [20], a GCN layer updates each file's representation by averaging its neighbors' messages, weighted by the inverse square root of the endpoint degrees under each relation:

$$h_i^{(k+1)} = \text{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \frac{1}{\sqrt{d_i^r d_j^r}} W^{r,(k)} h_j^{(k)} \Big), \tag{2}$$

where $W^{r,(k)}$ is the relation- and layer-specific transformation, and d_i^r and d_j^r denote the number of neighbors of files i and j in $\mathcal{N}_i^{r,\pi}$ and $\mathcal{N}_j^{r,\pi}$, respectively, so the degree follows whichever propagation scheme π is in effect. For forward propagation, these are incoming degrees, including the self-loop; for reverse propagation, they are outgoing degrees, including the self-loop. The implementation also learns a bias per relation and layer, omitted here for readability. ReLU [28] is the non-linear activation. The degree normalization prevents high-degree files, such as widely used base types, from dominating the aggregation.

GAT Following Veličković et al. [36], GAT replaces fixed degree-based weighting with learned attention coefficients. For each relation r, the features of each neighboring file are projected using $W^{r,(k)}$, scored, and normalized with a softmax over $\mathcal{N}_i^{r,\pi}$ to obtain the attention weight α_{ij}^r. The layer update for file i is:

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \alpha_{ij}^r \, W^{r,(k)} h_j^{(k)} \Big). \tag{3}$$

Equation 3 shows a single head; the implementation concatenates four heads per relation before summing relation outputs and applying ReLU. Learned attention weights increase flexibility but introduce additional parameters. Changing the propagation scheme π changes both the aggregated neighbors and their degree-normalization weights in GCN, whereas in GAT it changes the neighborhood over which attention is normalized.

3.3.2. Multi-layer Perceptron (MLP)

To assess the contribution of graph-neighborhood information, we use an MLP that operates on x_i alone and does not aggregate messages from other files. For each file i, the layer update rule is defined as:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big). \tag{4}$$

Unlike the GNNs, whose representations incorporate information from neighboring files over multiple dependency hops, the MLP constructs each representation from the file's own node features only. Performance differences between the MLP and the GNNs are therefore consistent with the contribution of dependency-graph information, though they should not be interpreted as isolating graph aggregation alone, since GAT's attention mechanism introduces additional parameters that the other two encoders do not have.

3.3.3. Classification head

All three encoders end in the same kind of output stage. The file embedding $z_i \in \mathbb{R}^d$, which is the output of the last message-passing layer for GCN and GAT and of the hidden layer for the MLP, is passed to a single linear layer that maps it to one score per architectural module. A softmax then converts these scores into a probability distribution over the modules:

$$\begin{aligned} p_i &= \mathrm{softmax}\big(W_c z_i + b_c\big), \\ \hat{y}_i &= \arg\max_m p_{i,m}, \qquad c_i = \max_m p_{i,m}, \end{aligned} \tag{5}$$

where $W_c \in \mathbb{R}^{|M|\times d}$, $b_c \in \mathbb{R}^{|M|}$, and $|M|$ is the number of architectural modules. The module with the highest probability, $\hat{y}_i$, is the model's prediction for file i, and its probability c_i is the confidence that the self-training procedure in Section 3.4 compares against the threshold τ. The same softmax output enters the cross-entropy loss during training.

3.4. Iterative Learning Framework

GCN and GAT receive the same dependency graph and node features, differing only in how they aggregate and weight messages from neighboring files; the MLP receives the node features alone and does not use the dependency graph. Despite these differences, all three models follow the same self-training procedure. Each produces a file embedding z_i, which the shared classification head maps to a probability over the architectural modules; the highest probability is the model's confidence in its predicted module for file i.

Training begins with a fixed set of labeled files L, while the remaining files form the unlabeled set U. After an initial warm-up phase on L, each round scores every file. Pseudo-labels whose current prediction disagrees with them and exceeds the confidence threshold are revised, and files from U whose confidence exceeds the threshold τ are promoted into the mapped set $\mathcal{M}$ with their current highest-probability predictions as labels. Ground-truth seed labels never change. The classifier is then trained further on $L \cup \mathcal{M}$ using full-batch cross-entropy, retaining the model parameters and Adam optimizer state across rounds. The procedure stops before further training when no new confident files are found, or after the maximum number of rounds is reached. Algorithm 1 summarizes this loop, identical across encoders except for the input representation and forward computation.

Algorithm 1: Iterative Mapping Algorithm.

Input: Input representation $\mathcal{I}$, files $\mathcal{F}$, labeled files L, confidence threshold τ, warm-up epochs E_w, self-training epochs E_s, max rounds R
Output: Trained classifier f, mapped files $\mathcal{M}$

```
M ← L; U ← F \ L;
Train f on (I, M) for E_w epochs;
for k ← 1 to R do
    Score all files in F with f using I;
    Revise any pseudo-label in M \ L whose confidence
      > τ disagrees with f;
    P ← {o ∈ U | confidence(o) > τ};
    if P = ∅ then
        break;
    Assign each o ∈ P its current highest-probability
      module as a pseudo-label;
    M ← M ∪ P; U ← U \ P;
    Retrain f on (I, M) for E_s epochs;
return f, M;
```

4. Experiments

This section describes the experimental design used to answer the three research questions: the benchmark systems, configuration, baseline, and evaluation metrics.

4.1. Benchmark Systems

We rely on existing GT mappings rather than constructing new ones, since producing a reliable C2A mapping demands substantial architectural knowledge, manual effort, and often expert validation [13]. Table 2 lists the ten Java and C/C++ systems used in our evaluation; the file, module, and edge counts reflect the fixed-split data actually used during training and evaluation, which for some systems excludes a small number of files or edges present in the raw pre-processed dataset. We also report file-level dependency homophily $\mathcal{H}$, the fraction of static dependency edges whose source and target files belong to the same ground-truth module, ranging from 0 (every dependency crosses modules) to 1 (every dependency stays within one). For homophily, each ordered pair of files with at least one static dependency contributes once, regardless of dependency type. The edge count $|E|$ instead counts distinct (source file, target file, dependency type) triples, matching the multi-relational graph used by the encoders. Thus, homophily is computed on the merged directed graph, whereas the reported average degree counts typed edges.

The Java systems are drawn from the same source as our earlier work [14], assembled by Olsson et al. [29] from systems previously studied at SAEroCon and by Brunet et al. [4] and Lenhard et al. [22], with mappings distributed through the s4rdm3x GitHub repository[2].

For Bash, we use the C2A mapping curated by Garcia et al. [13], produced by deriving an initial decomposition from documentation and source code and then validating it with system experts. The same line of work provides the GT mapping for Chrome, established through roughly two years of collaboration with the system's developers [24].

The GT mappings for HDF and HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [40], distributed through the SARIF replication package[3].

4.2. Experimental Configuration

GCN and GAT use two message-passing layers; the one-layer ablation is examined separately in Section 5. Both use 256 hidden channels. GAT concatenates four attention heads with 64 channels per head at each layer. The MLP has one hidden layer with 256 units. GCN and GAT apply dropout of 0.01 to node activations after each ReLU; the MLP uses no dropout. All three encoders are followed by the same kind of linear classification head. Thus, the encoders share the features, classifier form, and self-training schedule, but are not matched in depth or parameter count. All three models are trained with the Adam optimizer at a learning rate of 0.001, minimizing cross-entropy loss over the architectural modules on the labeled and pseudo-labeled files.

Table 2
Properties of the software systems. C denotes the number of modules in the GT architecture, edges are distinct typed dependencies between files, and the average typed out-degree is computed as $\langle k \rangle = |E|/|\text{Files}|$.

	System	Version	LoC	C	|Files|	$\|E\|$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	r13713	62k	14	766	9 491	0.55	12.4
	C.Img	v1.0a2	18k	21	329	4 561	0.61	13.9
	JabRef	v3.7	59k	6	1 180	17 557	0.59	14.9
	Lucene	r1075001	36k	7	1 054	18 595	0.68	17.6
	SH-3D	r002382	35k	9	167	3 912	0.36	23.4
	T.Mates	v5.11	55k	15	778	17 818	0.34	22.9
C/C++	Bash	4.2	115k	13	292	1 222	0.60	4.2
	HDF	0e196f	153k	9	153	1 239	0.35	8.1
	HDC	46ff87	26k	11	207	1 117	0.63	5.4
	Chrome	svn-171054	11M	69	18 343	284 435	0.58	15.5

All models use the same iterative learning procedure: a warm-up phase of $E_w = 100$ epochs on the initial labeled set, followed by at most $R = 4$ self-training rounds of $E_s = 30$ epochs each. The initial labeled set contains 5% of a system's files, bounded between 30 and 200 files. The lower bound provides sufficient supervision in small systems, where 5% alone would yield too few labeled files, while the upper bound limits the manual labeling effort in large systems such as Chrome. After sampling, we add one randomly selected file from each module still missing from the initial labeled set, so every module has at least one labeled file before training. During self-training, only predictions with confidence above $\tau = 0.95$ are added as pseudo-labels. These settings are fixed across the neural models. We use the first 100 supplied fixed splits, with run IDs 0–99, for all methods and ablations. The initial 200-file cap applies before adding representatives of missing modules; the final seed set can therefore exceed 200 files.

The neural models were trained on an NVIDIA GB10 GPU using Python 3.11, PyTorch 2.9.1, and PyTorch Geometric 2.7.0; NBA ran on the CPU. Node features were generated once per system and reused across runs and neural configurations. The training seed was set to $42 + r$ for split r, independently of feature generation. The complete software environment, configurations, and split identities are recorded in the replication package.

To examine sensitivity to initialization and training randomness, we also conducted an exploratory follow-up using the first ten splits crossed with five training seeds, keeping the features fixed. The selected comparisons cover close model rankings and depth or direction effects, with the large Chrome graph advantage included as a reference. A subset was repeated under deterministic GPU execution after a same-seed discrepancy was observed on A.UML. These follow-ups are reported separately from the primary 100-run experiment.

[2] https://github.com/h0bb3/s4rdm3x
[3] https://github.com/anonymous2f4a9d/SARIF_FSE23

4.3. Baseline: Naive Bayes

We compare against *NBAttract* [29] (NBA), a multinomial Naive Bayes classifier originally proposed for semi-supervised code-to-architecture mapping. NBA represents each entity using text features extracted from code identifiers and package names, combined with module-level Concrete Dependency Abstraction (CDA) sentences, document terms that encode cross-module structural relations, e.g., *A-Inherits-B* when an entity in module *A* inherits from one in module *B*. Olsson et al. report that NBA outperforms *CountAttract* [6] and the information retrieval methods *IRAttract* and *LSIAttract* [3] on the same benchmark [29], so we treat NBA rather than those earlier methods as the reference point for prior work.

NBA was originally proposed and evaluated on Java systems only; we extend it to C/C++ systems in our benchmark and re-implement the algorithm in Python, following the original iterative procedure, including its 0.90 confidence threshold for adding pseudo-labels and the same initial labeled set used by the other methods. Self-training stops either when no remaining entity can be mapped with sufficient confidence or when all entities have been mapped. Code for our NBA re-implementation and C/C++ extension is available in the GitHub repository 1. For the Python baseline, Java node names retain the qualified entity names supplied by the extractor; for C/C++, directory-qualified file base names provide the analogous naming context. Identifier tokens use English Snowball stemming. We use binary term occurrences, multinomial Naive Bayes with Laplace smoothing ($\alpha = 1$), and class priors from the currently mapped files. Architectural module names also contribute training documents. This is a Python adaptation using scikit-learn, not a claim of numerical equivalence to the original Java/Weka implementation.

4.4. Performance Evaluation

Module distributions in the benchmark systems are highly imbalanced, with several modules containing only one or two entities. We use macro-F1 to expose performance on smaller modules, which overall accuracy can obscure. We also report micro-F1, which equals accuracy for this single-label, exhaustive classification task. F1 is the harmonic mean of precision and recall. Precision measures the proportion of entities assigned to a module that actually belong to it, whereas recall measures the proportion of entities that truly belong to a module and are correctly assigned.

We report macro- and micro-averaged F1 scores. Macro-F1 is the unweighted average of per-module F1 scores over modules represented in the evaluated ground-truth subset. A module whose files all belong to the seed set has no held-out examples and is excluded from that macro average. Undefined precision and recall are assigned zero. Micro-F1 considers all predictions, including predictions into modules absent from the test ground truth, and reflects overall file-level mapping performance.

5. Results

We report results from 100 runs per model and system, using the same fixed labeled sets for all methods and ablations. All primary metrics cover the full test set, excluding the initial labeled files. For the neural models, each test file receives the final model's prediction, whether or not it was confidently mapped during self-training. NBA retains accepted assignments and assigns the remaining files using their last attraction scores. Thus, no method benefits from evaluation on a smaller subset. Results are summarized as the median across the 100 runs. This section addresses the three research questions in order.

We use two-sided Wilcoxon signed-rank tests [37] to assess statistical significance, pairing runs by their shared split ID. The compared neural configurations also share the training seed within each split. We apply Holm–Bonferroni correction at $\alpha = 0.05$, grouping comparisons by the question being tested: neural methods (60 tests), neural methods against NBA (60), relation types (40), depth (40), and direction (80). Each family includes both metrics and all ten systems. The tables report differences between the two medians, not medians of paired differences. Cliff's δ [8] describes the distributional effect size, classified by magnitude as negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), or large ($|\delta| \geq 0.474$) [31].

RQ1. Overall Effectiveness of C2A Mapping

To answer RQ1, we compare NBA_{CDA}, MLP, GCN, and GAT over 100 runs per system, all initialized from the same labeled set; NBA_{CDA} follows its own self-training procedure (Section 4.3), while MLP, GCN, and GAT share the iterative learning loop of Section 3.4. Table 3 reports median macro and micro F1. All three neural methods outperform the Python NBA adaptation on both metrics across all ten systems, with statistically significant differences, although the size of the advantage varies. For example, MLP exceeds NBA by only 2.1 micro-F1 points on JabRef. We focus the remaining comparison on MLP, GCN, and GAT, with MLP assessing the contribution of node features without exploiting the dependency graph.

Using unrounded medians, GAT achieves the highest macro F1 on six systems and micro F1 on eight. MLP has the highest macro F1 on A.UML, SH-3D, HDF, and HDC, although several differences are small. On micro F1, MLP leads on HDF and GCN on HDC. HDF and HDC are among the smallest and least connected benchmark systems, whereas SH-3D is comparatively well connected but has low modular cohesion; in these systems, MLP either outperforms GAT or remains competitive. On Chrome, the only large-scale system in the benchmark, GAT and GCN exceed MLP by 21.8 and 19.0 macro-F1 points, and by 17.1 and 16.1 micro-F1 points, respectively. Because Chrome is the only large-scale system, this result alone cannot establish whether the graph advantage generalizes as system size increases. GCN consistently outperforms NBA but does not occupy a consistent position between MLP and GAT. RQ2 examines these system-level differences.

Table 3
Median F1 scores (×100) across 100 runs. MA and MI denote macro- and micro-F1, respectively. The highest unrounded median for each system–metric pair is highlighted; highlighting does not imply statistical significance.

	A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA_{CDA}	32.6	64.7	17.5	47.7	47.2	89.7	60.6	82.3	39.6	73.5	32.5	68.8	27.6	54.0	33.4	48.8	9.5	54.4	44.0	76.7
MLP	82.8	89.6	90.3	93.6	66.1	91.7	85.3	95.0	84.8	94.9	92.4	95.4	64.7	82.3	89.5	90.1	59.2	72.4	76.4	86.7
GCN	78.8	91.3	90.7	93.8	80.0	94.1	85.5	93.7	78.5	91.8	79.5	91.3	68.2	87.5	79.6	82.9	78.3	88.5	70.8	88.8
GAT	82.5	93.2	91.5	95.5	85.1	95.8	89.8	95.1	84.7	95.6	94.7	97.1	69.4	87.9	81.9	83.6	81.1	89.5	74.5	88.6

The same encoder and training settings are used for Java and C/C++ systems, with the language-dependent naming and path handling described in the feature construction. Bash, HDF, and HDC are the smallest and sparsest systems in the C/C++ subset, and MLP is competitive with or outperforms the GNNs on two of them.

Statistical significance of observed differences. Table 4 reports pairwise statistical comparisons of the three learned methods for the performance differences shown in Table 3. GAT significantly outperforms GCN on macro F1 for nine systems and on micro F1 for seven, with effects ranging from small to large. Neither difference is significant on Bash; on HDC, only the macro-F1 advantage is significant. GCN's difference from MLP varies substantially by system and metric. MLP significantly outperforms GCN on macro F1 on A.UML, SH-3D, T.Mates, HDF, and HDC, whereas GCN significantly outperforms MLP on JabRef and Chrome on both metrics. On Bash, GCN does not significantly outperform MLP on macro F1 but does on micro F1, with a large effect.

Table 4
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor the first method. Cells in which the second method is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT − GCN		GCN − MLP		GAT − MLP	
System	MA	MI	MA	MI	MA	MI
A.UML	+3.7 (M)	+1.9 (L)	−4.0 (M)	+1.7 (S)	−0.3† (N)	+3.7 (L)
C.Img	+0.7 (S)	+1.7 (L)	+0.4† (N)	+0.2† (N)	+1.1 (S)	+1.9 (L)
JabRef	+5.0 (M)	+1.7 (L)	+13.9 (L)	+2.3 (L)	+19.0 (L)	+4.0 (L)
Lucene	+4.3 (M)	+1.4 (M)	+0.3† (N)	−1.3 (M)	+4.5 (M)	+0.0† (N)
SH-3D	+6.2 (M)	+3.8 (L)	−6.3 (L)	−3.1 (L)	−0.2† (N)	+0.7† (N)
T.Mates	+15.2 (L)	+5.9 (L)	−12.9 (L)	−4.1 (L)	+2.3 (S)	+1.8 (M)
Bash	+1.2† (N)	+0.4† (N)	+3.5† (S)	+5.2 (L)	+4.7 (S)	+5.6 (L)
HDF	+2.2 (S)	+0.7† (N)	−9.8 (L)	−7.2 (L)	−7.6 (L)	−6.5 (L)
Chrome	+2.8 (L)	+1.1 (S)	+19.0 (L)	+16.1 (L)	+21.8 (L)	+17.1 (L)
HDC	+3.6 (S)	−0.2† (N)	−5.5 (S)	+2.0† (N)	−1.9† (N)	+1.9† (N)

GAT significantly outperforms MLP on macro F1 for C.Img, JabRef, Lucene, T.Mates, Bash, and Chrome, and on micro F1 for A.UML, C.Img, JabRef, T.Mates, Bash, and Chrome. The differences on SH-3D are not significant on either metric. In contrast, MLP significantly outperforms GAT on both metrics only for HDF, with large effects. Despite its low connectivity, Bash favors GAT on both metrics; this exception is discussed in RQ2.

How stable is mapping across runs? Each of the 100 runs uses a different labeled partition and training seed; Figure 2 therefore shows F1-score variability arising from both sources. Micro F1 is generally stable, with interquartile ranges below 6 points for seven of ten systems. Macro F1 varies more because changes in the initial labeled set disproportionately affect small modules. This pattern is shared across MLP, GCN, and GAT. HDC is the most variable system, with macro-F1 interquartile ranges of 13–14 points, compared with about 8 points for micro-F1.

Variability is associated with the number of initially labeled files and, for some systems, graph connectivity, rather than system size or language. HDC, HDF, Bash, and SH-3D start near the 30-file floor, leaving only one or two labeled files per module. HDC, HDF, and Bash are also sparse, further limiting the structural signal available to graph-based models, whereas SH-3D is comparatively well connected. C.Img provides a contrast: despite its small size, it is dense and cohesive and remains stable. Once the initial training set reaches roughly 50 files, interquartile ranges are generally narrow regardless of scale; C.Img and Chrome are among the most stable systems on macro F1. Remaining macro-F1 variability is associated with module-size skew: systems with both very large and small modules, such as JabRef, retain wider distributions despite adequate supervision. RQ3 fixes the labeled partition across repeated runs to isolate the contribution of the training seed.

Answer to RQ1.

All three neural methods outperform NBA_{CDA} on both metrics across all ten systems. GAT achieves the highest averages of system-level median macro F1 (83.5) and micro F1 (92.2), leading on six and eight systems, respectively. GCN improves over NBA_{CDA} but does not consistently outperform the graph-free MLP baseline. Run-to-run variability is largest on macro F1 for the smallest systems. RQ2 examines the graph contribution beyond lexical features, while RQ3 isolates variability from the training seed.

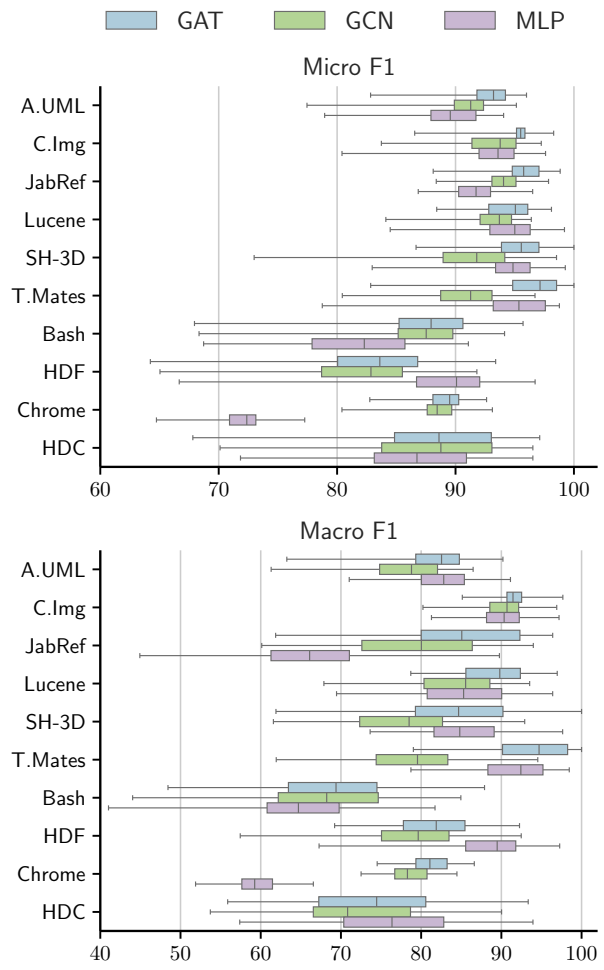


Figure 2: Distribution of micro F1 (top) and macro F1 (bottom), in points, across 100 runs for each model and system. Boxes show the median and interquartile range; whiskers span the observed range.

RQ2. Contribution of the Dependency Graph

RQ1 shows that no learned method dominates throughout the benchmark. RQ2 examines what the dependency graph and GNNs contribute beyond lexical features: which system properties are associated with the benefit of message passing, and how often each model correctly classifies individual files across runs, a question median F1 cannot answer since it aggregates performance at the system level. All three neural models use identical lexical features and training loops, although their encoder architectures and capacities differ.

Which system properties are associated with where the graph helps? Connectivity describes how much information is available from neighboring files; homophily measures how often dependencies connect files in the same ground-truth module [41]. Bash, HDC, and HDF have the lowest average typed out-degrees in the benchmark (4.2, 5.4, and 8.1, respectively). MLP remains competitive with GAT on HDC and significantly outperforms it on HDF, but GAT significantly outperforms MLP on Bash. Sparse connectivity alone therefore does not determine whether message passing helps. These systems also differ in homophily, module composition, and the supervision available per module.

On the low-homophily systems SH-3D ($\mathcal{H} = 0.36$) and T.Mates ($\mathcal{H} = 0.34$), GCN performs significantly worse than MLP with large effect sizes. GAT is more competitive: neither difference from MLP is significant on SH-3D, while T.Mates favors GAT by 2.3 macro-F1 and 1.8 micro-F1 points. One possible explanation is that attention can assign less weight to uninformative cross-module neighbors than degree-normalized aggregation. We did not measure this mechanism directly, however, and the crossed follow-up shows seed-dependent rankings on both systems. On HDF ($\mathcal{H} = 0.35$), both GNNs perform significantly worse than MLP. Together, these observations are compatible with connectivity and homophily influencing the value of message passing, but do not establish a rule for selecting an encoder.

Which files does the graph help most, and where does it fail? For each file, we compute its misclassification rate as the proportion of its test-set evaluations in which its predicted module differs from the ground-truth module, and refer to files with a rate of at least 0.9 as *persistently wrong*. Files that appear only in labeled sets are excluded. Figure 3 contrasts Chrome against the pooled distribution of the other nine systems, giving each evaluated file equal weight.

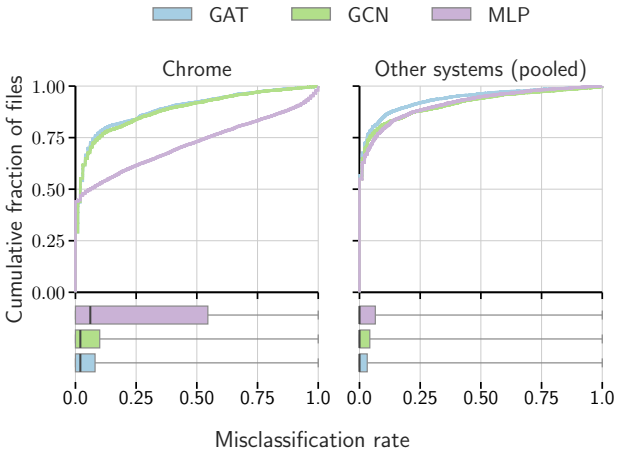


Figure 3: Empirical cumulative distribution of per-file misclassification rates across 100 runs, for GAT, GCN, and MLP, on Chrome (left) and pooled across the remaining nine systems (right). The y-axis is the cumulative proportion of test-set files whose misclassification rate is at or below the value on the x-axis, so a curve that rises earlier and higher indicates more reliable per-file decisions. The box plot below each panel shows the median, interquartile range, and full range of the misclassification rate for each model.

Across the nine small-to-medium-sized systems, the pooled distributions are concentrated near low error rates for all three models. Chrome shows a much larger contrast: MLP has a wider misclassification-rate distribution and a heavier tail of files misclassified in nearly every run. Among

the 18,342 Chrome files evaluated at least once, 1,913 are persistently wrong under MLP (10.4%), compared with 213 under GCN (1.2%) and 204 under GAT (1.1%). Thus, the graph advantage on Chrome is visible not only in aggregate F1 but also in the frequency of repeated file-level errors.

Across all ten systems, 241 files meet the persistent-error criterion under default GAT, of which 204 are in Chrome. These counts describe the current 100-run experiment; they can change with the labeled sets, training seeds, and error threshold. RQ3 examines how the persistent-error counts change when edge direction is altered.

Answer to RQ2.

Connectivity and homophily provide plausible context for the system-level differences, but neither alone determines whether a GNN improves on lexical features. GAT is more competitive than GCN on the low-homophily systems SH-3D and T.Mates, while MLP significantly outperforms both GNNs on HDF. On Chrome, both GNNs substantially improve aggregate F1 and reduce the share of persistently wrong files relative to MLP. These results support a contribution from dependency information, without isolating it from encoder capacity or establishing a causal model-selection rule.

RQ3. Sensitivity to Design Choices

RQ1 and RQ2 establish where the approach performs well and what the dependency graph contributes. RQ3 examines whether modeling dependency types separately is necessary and how sensitive the approach is to two message-passing design choices: the number of layers and the direction of aggregation. A crossed follow-up then tests whether the training seed alone can change the rankings observed in RQ1 and the ablation comparisons above. Because the benchmark systems have highly imbalanced module sizes, macro F1 can be volatile: it gives each module equal weight, so reassigning only a few files in a small module can substantially change the score. Micro F1 instead weights each file equally and may remain largely unchanged by the same reassignments. We therefore report both metrics throughout and regard a significant difference in either metric as evidence that the design choice affects performance.

Does modeling dependency types separately matter? Our encoders keep one adjacency matrix per relation type and learn a separate transformation for each, rather than collapsing all dependencies into a single edge type. Figure 4 compares this multi-relational design against a uni-relational variant, in which all dependency types are merged into one undifferentiated adjacency matrix before message passing, for both GAT and GCN across the ten benchmark systems.

Both GAT and GCN perform significantly better with multiple relation types than with merged edges on every system and both metrics. Median macro-F1 gains range from 6.6 to 40.2 points for GAT and from 8.1 to 32.7 points for GCN. The smallest gain occurs on C.Img for both encoders, and the largest on T.Mates. This is consistent with different

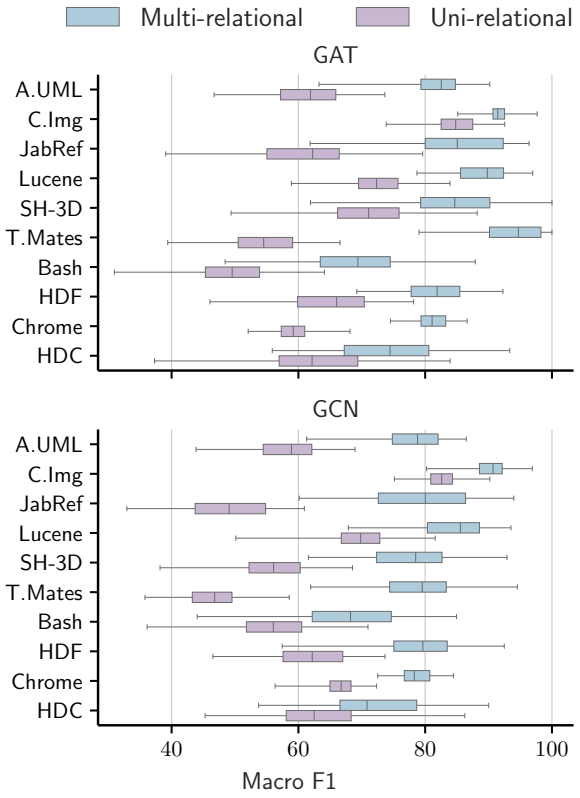


Figure 4: Distribution of macro F1, in points, across 100 runs for the multi-relational and uni-relational variants of GAT and GCN. Boxes show the median and interquartile range; whiskers span the observed range.

dependency types carrying different structural signals about module membership: a call edge and an inheritance edge need not carry the same evidence that two files share a module. The multi-relational encoder also has more parameters than the uni-relational one, so the experiment does not isolate relation typing from added capacity.

How many message-passing layers does the graph support? Additional GNN layers enlarge each node's receptive field, which is the L-hop neighborhood that can influence its representation after L message-passing layers. However, when repeated aggregation mixes information from a large portion of the graph, node representations can become increasingly similar and lose the discriminative information needed for classification, an effect known as over-smoothing [23]. We therefore explore only one and two layers, and Table 5 reports the comparison. According to Table 5, two layers significantly improve GAT's macro F1 on JabRef, T.Mates, Bash, and Chrome, and its micro F1 on JabRef, Bash, and Chrome. They significantly reduce macro F1 on HDC (-6.0 points) and micro F1 on Lucene (-1.1 points). Neither difference is significant on SH-3D. For GCN, two layers significantly improve both metrics on JabRef and Bash, and micro F1 on HDF and Chrome. They significantly reduce micro F1 on C.Img and SH-3D, both metrics on T.Mates, and macro F1 on HDC. The HDF

Table 5
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor two layers; cells in which one layer is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT (2L − 1L)		GCN (2L − 1L)	
System	MA	MI	MA	MI
A.UML	+0.4† (N)	−0.4† (N)	−0.3† (N)	−0.6† (N)
C.Img	+0.1† (N)	+0.0† (N)	−0.4† (N)	−1.1 (S)
JabRef	+3.2 (S)	+1.3 (M)	+5.1 (M)	+1.6 (M)
Lucene	+2.4† (N)	−1.1 (S)	+2.8† (S)	−0.5† (N)
SH-3D	+0.9† (N)	+0.7† (N)	+0.0† (N)	−0.9 (S)
T.Mates	+2.4 (S)	−0.3† (N)	−5.6 (L)	−2.6 (L)
Bash	+5.0 (M)	+4.6 (L)	+6.1 (S)	+3.8 (M)
HDF	−2.0† (N)	−0.9† (N)	−0.1† (N)	+1.6 (N)
Chrome	+2.9 (L)	+3.0 (L)	+0.1† (N)	+1.3 (L)
HDC	−6.0 (S)	−0.5† (N)	−9.1 (M)	−0.2† (N)

micro-F1 gain does not remain significant when all neural comparisons are corrected together.

These results support system-dependent depth choices. The gains on JabRef are compatible with informative two-hop neighborhoods, while the losses on T.Mates for GCN are compatible with greater mixing across modules. These mechanisms were not measured directly, however, and neither connectivity nor homophily alone explains every result. Two GAT layers benefit several systems in this benchmark, but are not uniformly preferable.

Does edge direction matter? The heterogeneous encoder aggregates source-to-target by default, so a file's representation is shaped by the neighbors that depend on it rather than by the neighbors it depends on. Table 6 compares this against an undirected variant, which symmetrizes every edge, and a reverse variant, which aggregates target-to-source. As shown in Table 6, source-to-target aggregation significantly outperforms the reversed version in macro F1 across six systems: C.Img, JabRef, SH-3D, T.Mates, Chrome, and HDC for GAT. Reversing the aggregation notably enhances macro F1 in A.UML and Lucene by 1.5 and 4.2 points, respectively, and raises micro F1 in Lucene by 1.2 points. No significant difference is observed between original and reversed aggregation on Bash or HDF. Compared to undirected aggregation, the directed default significantly improves at least one metric on nine systems, although no difference is significant on Bash.

GCN is more mixed: the default has a significant advantage over at least one alternative on six systems, and a significant disadvantage on six, with some systems in both groups. On JabRef, reversal reduces macro F1 but improves micro F1; the latter difference is not significant under the combined correction across all neural comparisons. These metric-dependent effects do not support one direction for every system.

At the file level, reversal reduces the number of persistently wrong GAT files from 11 to 6 on Lucene and leaves it at 9 on A.UML. Chrome illustrates a different trade-off: reversal reduces the count from 204 to 103 even though median macro F1 falls by 4.3 points; undirected aggregation raises the count to 322. Aggregate F1 and the size of the persistent-error tail can therefore move in different directions. These counts do not establish that the same individual errors are resolved without introducing new ones.

How robust are these rankings to the training seed? The 100-run comparisons in RQ1 and the preceding ablations vary both the labeled partition and the training seed. We therefore conduct an exploratory follow-up to test whether changing only the training seed can alter model rankings. For each of ten fixed splits, we train each configuration with five seeds. We summarize each comparison using the mean paired difference, first averaging across seeds within each split. To quantify uncertainty, we use a crossed bootstrap that resamples splits and seeds separately. We apply a Bonferroni adjustment to the 95% intervals for the 24 comparisons in the broader follow-up and retain this adjustment for the deterministic subset. Thus, the 50 runs per configuration are not treated as 50 independent splits.

Table 7 reports the selected comparisons repeated under deterministic execution. Even with the labeled files held fixed, the macro-F1 ranking changes across seeds on six of ten splits for GAT versus MLP on SH-3D, five for the same comparison on T.Mates, and eight for GAT versus GCN on HDC. None of these three intervals rules out the possibility of no difference. This does not overturn RQ1's significant advantages on T.Mates and HDC, but it shows those advantages are less consistent once the effect of seed is isolated. For the reversal comparison, Lucene holds up under this check, while A.UML does not. GAT's advantage over MLP on Chrome is the most robust result: it wins in all 50 split-seed combinations, with a large and consistent margin of about 22.6 points and an adjusted interval of [19.1, 26.6].

The deterministic follow-up was prompted by an A.UML run that changed when repeated with the same seed. Deterministic execution produced identical results in all twelve tested configuration-level repeat checks. However, it changed the selected mean comparisons by less than 0.1 F1 points and left all seed-ranking flip counts unchanged. Thus, the observed seed sensitivity cannot be explained by numerical nondeterminism alone.

Answer to RQ3.

Multi-relational aggregation improves both metrics on every system for both encoders, although part of the gain may reflect added capacity. Depth and direction are more system-dependent: two GAT layers benefit several systems, while reversed aggregation improves macro F1 on A.UML and Lucene. The deterministic follow-up supports the Lucene and Chrome advantages but shows that several closer rankings, on SH-3D, T.Mates, and HDC, depend on the training seed. No depth or direction is uniformly best, and direction changes can reduce persistent errors even when aggregate macro F1 decreases, as on Chrome.

Table 6
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI) under default directed (D) source-to-target aggregation, compared with UD (undirected, symmetrized edges) and RD (reversed, target-to-source), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor D; cells where the alternative is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT				GCN			
	D − UD		D − RD		D − UD		D − RD	
System	MA	MI	MA	MI	MA	MI	MA	MI
A.UML	+2.7 (S)	+1.5 (M)	−1.5 (S)	−0.2† (N)	−0.9 (S)	−1.0 (S)	−3.3 (M)	−1.5 (M)
C.Img	+2.9 (L)	+1.4 (L)	+5.8 (L)	+2.1 (L)	+3.8 (M)	+0.3† (S)	+9.3 (L)	+1.1 (S)
JabRef	+10.7 (L)	+1.4 (M)	+8.3 (L)	+0.2† (N)	+3.6† (N)	−0.3† (N)	+10.6 (L)	−0.8 (S)
Lucene	+3.1 (M)	+1.2 (S)	−4.2 (L)	−1.2 (M)	−2.0† (S)	−0.6 (S)	−7.4 (L)	−1.4 (M)
SH-3D	+18.6 (L)	+7.4 (L)	+14.5 (L)	+5.9 (L)	+8.7 (L)	+3.6 (L)	+16.6 (L)	+5.2 (L)
T.Mates	+8.2 (L)	+1.4 (S)	+9.5 (L)	−0.1† (N)	−0.9† (S)	−2.3 (M)	+0.8† (N)	−4.8 (L)
Bash	−1.6† (N)	+0.4† (N)	−1.5† (N)	+0.1† (N)	−3.2 (S)	−0.7† (N)	−3.7 (S)	−0.4† (N)
HDF	+3.6 (M)	+4.2 (L)	−3.0† (S)	−1.5† (N)	+3.1 (S)	+4.4 (M)	−2.5 (S)	+0.2† (N)
Chrome	+6.2 (L)	+2.0 (L)	+4.3 (L)	−0.1† (N)	+2.4 (M)	−0.1† (N)	+2.6 (L)	−0.1† (N)
HDC	+8.9 (M)	+1.3 (N)	+10.9 (L)	−0.2† (N)	+3.1 (S)	+0.8† (N)	+11.1 (L)	+1.8† (N)

Table 7
Exploratory macro-F1 comparisons under deterministic execution. Effects are mean paired differences in F1 points, with family-adjusted 95% crossed-bootstrap intervals. *Flips* counts the number of splits in which both positive and negative differences occur across the five seeds. RD and D denote reversed and default directions.

System	Comparison	Mean	Interval	Flips
A.UML	GAT RD − D	+2.4	[−1.9, 6.4]	3/10
SH-3D	GAT − MLP	+1.0	[−5.2, 7.0]	6/10
T.Mates	GAT − MLP	+2.1	[−2.7, 6.9]	5/10
HDC	GAT − GCN	+4.5	[−1.1, 11.3]	8/10
Lucene	GAT RD − D	+6.7	[2.1, 11.8]	3/10
Chrome	GAT − MLP	+22.6	[19.1, 26.6]	0/10

6. Threats to Validity

External Validity: The ten benchmark systems span Java, C, and C++, but they do not represent the full range of architectural styles, application domains, development practices, or programming languages. They were selected because usable ground-truth mappings were available. Chromium is the only system at industrial scale; consequently, observations about scalability rest on a single system rather than a broader trend. None of the systems consists primarily of AI-generated code, so it remains unknown whether such code yields dependency structures and module cohesion comparable to those in the studied systems.

Our approach is transductive and assumes that the graph is fixed during training. Adding files or changing dependencies requires reconstructing the graph and retraining the model. The approach is therefore more suitable for periodic architecture analyses and maintenance workflows than for incremental mapping at commit time.

Internal Validity: The dependency graph is derived through static analysis. Static extraction can miss dependencies that arise only at runtime, for example through reflection, function pointers, conditional compilation, or macro expansion. It can also introduce imprecise edges when references cannot be resolved unambiguously. Because message passing depends on the extracted graph, missing and incorrect edges can directly affect the model's predictions.

A further threat arises from version alignment between the analyzed source code and the published ground-truth mapping. Several benchmark mappings were created years before the source versions used in this study were obtained. In addition, the files included in a system can depend on its build configuration. The exact file set in the analyzed revision may therefore differ from that represented in the ground-truth mapping.

Files can plausibly belong to more than one module, and different annotators may assign such files differently. In older systems, architectural drift may also weaken the correspondence between a file and its assigned ground-truth module. Since the initial labeled set is sampled from the same mapping used for evaluation, an incorrect ground-truth assignment can both provide misleading supervision during training and penalize a prediction that is architecturally plausible.

The primary unit of analysis is the file, following the granularity of the available ground-truth mappings. A file that implements multiple concerns must nevertheless receive one module label. Neither the GNNs nor the compared baselines can represent multiple simultaneous module assignments.

The 100 initial labeled sets and training seeds are fixed and reused across neural configurations. Since both change between runs, the primary experiment does not separate their contributions to variability. The crossed follow-up holds each partition fixed while varying the training seed and reveals seed-dependent rankings on several systems. It covers only ten splits and five seeds, with comparisons

selected after the main experiment, and should therefore be interpreted as exploratory rather than as a replacement for the primary analysis.

The primary experiment did not enforce deterministic GPU operations. Same-seed controls exposed a numerical discrepancy on A.UML that was amplified during training and pseudo-label selection. Deterministic execution produced identical results in the tested repeat controls, but did not remove the seed-dependent rankings or materially change the selected average comparisons. This check does not establish repeatability on every software or hardware configuration.

Construct Validity: A fixed confidence threshold determines whether an unlabeled file receives a pseudo-label. This threshold captures how concentrated the model's softmax output is, rather than mapping quality alone. We evaluate predictions over the complete test set, preventing a model that pseudo-labels fewer files from benefiting from evaluation on an easier subset.

Directory paths are included in the node features, and in several benchmark systems the folder hierarchy closely reflects the reference module decomposition. Where this alignment is strong, performance may be optimistic because folder-path features provide partial information about the target module, rather than requiring the model to infer the mapping from identifiers and dependencies alone.

The encoders share features and a training schedule but differ in depth, dropout, and parameter count. In particular, the multi-relational GNNs have more parameters than their merged-relation counterparts. Their advantages cannot therefore be attributed solely to graph structure or relation typing. The NBA baseline is a Python adaptation extended to C/C++, not a numerically verified reproduction of the original Java implementation.

Statistical significance depends in part on the comparison family. Combining all neural comparisons into one correction family leaves most conclusions unchanged, but removes significance for GCN's micro-F1 depth gain on HDF and reversal gain on JabRef. Persistent-error counts also depend on the chosen threshold and the files' test-set appearances; they describe this experiment rather than a seed-invariant set of difficult files.

7. Conclusion

We studied code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, comparing GAT and GCN against an MLP baseline that receives the same lexical features and passes through the same self-training loop. Across ten systems in Java and C/C++ and 100 runs per system, all three neural models outperform the Naive Bayes reference on both metrics. GAT achieves the highest averages of system-level median macro F1 (83.5) and micro F1 (92.2), although MLP remains competitive or superior on several systems. On Chrome, the largest system in the benchmark at 18,343 files, GAT and GCN outperform MLP by 22 and 19 macro-F1 points, respectively. With only one large-scale system in the benchmark, this gap cannot establish whether the graph's advantage scales with system size more generally.

Two system properties are plausibly associated with where this advantage appears: connectivity sets a baseline for how much neighborhood information a file can aggregate, and homophily measures how often dependency edges connect files in the same ground-truth module. These associations are observational and do not establish a causal selection rule. At the file level, Chrome again shows the clearest contrast: MLP persistently misclassifies 10.4% of the files observed in a test set, compared with 1.2% for GCN and 1.1% for GAT.

Letting the encoder learn a separate transformation per relation outperforms a single merged edge type on every system, although this also increases model capacity. Depth and direction have more system-dependent effects. Two GAT layers benefit several systems but are not uniformly preferable, and reversed aggregation improves macro F1 on A.UML and Lucene. Direction can also trade aggregate performance against persistent errors: on Chrome, reversal reduces the persistent-error count despite a lower macro F1.

The crossed seed study supports the large graph advantage on Chrome and the reversal advantage on Lucene, while showing that several other rankings vary with the training seed. Deterministic execution removes discrepancies in the tested same-seed controls without resolving this sensitivity. Average advantages should therefore not be read as guarantees for an individual labeled set or training run.

Three directions could clarify and extend the contribution of dependency information. First, an additional attention mechanism over relation types could replace the unweighted sum of relation outputs when different dependencies provide conflicting evidence; the current GAT already learns separate neighbor-attention parameters for each relation. Second, the graph could incorporate finer-grained architectural information, including folder hierarchies and class members, to better reflect how architects conceptualize modules. Third, separating identifier-based lexical features from path-based features would clarify how much of the MLP baseline's signal comes from naming versus directory structure, and how much of that overlaps with what the dependency graph already encodes.

CRediT authorship contribution statement

Rakhshanda Jabeen: Conceptualization, Methodology, Software, Investigation, Writing (original draft, review & editing). **Morgan Ericsson:** Conceptualization, Software review, Supervision, Writing (review & editing). **Jonas Nordqvist:** Supervision, Writing (review & editing). **Anna Wingkvist:** Supervision, Writing (review & editing).

Declaration of use of AI-assisted technologies

We used various AI tools to review the manuscript and the source code used for the experiments. We used the

output from these tools to manually update the manuscript and fix any issues found in the source code. We take full responsibility for the content of the manuscript and the source code provided in the reproduction package.

Funding

Rakhshanda Jabeen was supported by the Industrial Graduate School "Data Intensive Applications," funded by the Knowledge Foundation, Sweden (grant no. 20190336).

The Knowledge Foundation had no role in the study design; collection, analysis, or interpretation of data; preparation of the manuscript; or decision to submit the article for publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The replication package, including source code, experimental splits, and results, is available at https://github.com/sse-lnu/GCN-SAEroCon25, together with instructions for obtaining the benchmark data and reproducing the experiments. Upon acceptance, the version corresponding to this article will be archived on Zenodo, and its DOI will be included in the final manuscript.

References

[1] Ali, N., Baker, S., O'Crowley, R., Herold, S., Buckley, J., 2018. Architecture consistency: State of the practice, challenges and requirements. Empirical Software Engineering 23, 224–258.

[2] Amalfitano, D., De Luca, M., Santilli, T., Pelliccione, P., Fasolino, A.R., 2025. Automated software architecture design recovery from source code using llms, in: European Conference on Software Architecture, Springer. pp. 73–89.

[3] Bittencourt, R.A., Santos, G.J.d., Guerrero, D.D.S., Murphy, G.C., 2010. Improving automated mapping in reflexion models using information retrieval techniques, in: 17th Working Conference on Reverse Engineering, pp. 163–172.

[4] Brunet, J., Bittencourt, R.A., Serey, D., Figueiredo, J., 2012. On the evolutionary nature of architectural violations, in: 2012 19th Working Conference on Reverse Engineering, IEEE. pp. 257–266.

[5] Chen, J., Wang, Z., Jiang, Y., Pang, J., Zhang, T., Pan, M., Sun, J., 2022. NEGAR: network embedding guided architecture recovery for software systems, in: 2022 29th Asia-Pacific Software Engineering Conference (APSEC), IEEE. pp. 367–376. doi:10.1109/apsec57359.2022.00049.

[6] Christl, A., Koschke, R., Storey, M.A., 2007. Automated clustering to support the reflexion method. Information and Software Technology 49, 255–274.

[7] Clements, P., Garlan, D., Little, R., Nord, R., Stafford, J., 2003. Documenting software architectures: views and beyond, in: 25th International Conference on Software Engineering, 2003. Proceedings., IEEE. pp. 740–741.

[8] Cliff, N., 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 114, 494–509. doi:10.1037/0033-2909.114.3.494.

[9] De Silva, L., Balasubramaniam, D., 2012. Controlling software architecture erosion: A survey. Journal of Systems and Software 85, 132–151. doi:10.1016/j.jss.2011.07.036.

[10] Ehsani, R., Rawal, S., Cai, Y., Chatterjee, P., 2026. Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development. ACM Transactions on Software Engineering and Methodology doi:10.1145/3820165.

[11] Florean, A., Jalal, L., Sinkala, Z.T., Herold, S., 2021. A comparison of machine learning-based text classifiers for mapping source code to architectural modules, in: Companion Proceedings of the 15th European Conference on Software Architecture. URL: https://ceur-ws.org/Vol-2978/saerocon-paper1.pdf.

[12] Fuchß, D., Liu, H., Hey, T., Keim, J., Koziolek, A., 2026. Exarch: Enabling architecture traceability by llm-based architecture component name extraction, in: SE2026, Gesellschaft für Informatik, Bonn. pp. 61–62. doi:10.18420/se2026_20.

[13] Garcia, J., Krka, I., Mattmann, C., Medvidovic, N., 2013. Obtaining ground-truth software architectures, in: 35th International Conference on Software Engineering (ICSE), IEEE. pp. 901–910. doi:10.1109/icse.2013.6606639.

[14] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2025. Graph convolutional networks for mapping source code entities to architectural modules, in: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE. pp. 472–481.

[15] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026a. GAER: Graph auto-encoders for unsupervised software architecture recovery. Proceedings of the ACM on Software Engineering 3, 4094–4115. doi:10.1145/3808187.

[16] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026b. Graph embeddings for software architecture recovery. Nordic Machine Intelligence 6, 20–36. URL: https://journals.uio.no/NMI/article/view/13153, doi:10.5617/nmi.13153.

[17] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026c. Graph representation learning for software architecture recovery., in: ESANN, pp. 457–462. doi:10.14428/esann/2026.ES2026-316.

[18] Johansson, N., Caporuscio, M., 2026. Mapping source code to software architecture using embedding models: Insights from an analysis of data properties. Information and Software Technology 198, 108200. doi:10.1016/j.infsof.2026.108200.

[19] Johansson, N., Caporuscio, M., Olsson, T., 2024. Mapping source code to software architecture by leveraging large language models, in: Software Architecture. ECSA 2024 Tracks and Workshops, Springer. pp. 133–149. doi:10.1007/978-3-031-71246-3_13.

[20] Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=SJU4ayYgl.

[21] Kruchten, P., Nord, R.L., Ozkaya, I., 2012. Technical debt: From metaphor to theory and practice. IEEE Software 29, 18–21. URL: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf.

[22] Lenhard, J., Blom, M., Herold, S., 2019. Exploring the suitability of source code metrics for indicating architectural inconsistencies. Software Quality Journal 27, 241–274.

[23] Li, Q., Han, Z., Wu, X.M., 2018. Deeper insights into graph convolutional networks for semi-supervised learning, in: Proceedings of the AAAI conference on artificial intelligence, pp. 3538–3545. doi:10.1609/aaai.v32i1.11604.

[24] Lutellier, T., Chollak, D., Garcia, J., Tan, L., Rayside, D., Medvidović, N., Kroeger, R., 2018. Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 44, 159–181. doi:10.1109/tse.2017.2671865.

[25] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems 26.

[26] Murphy, G.C., Notkin, D., Sullivan, K., 1995. Software reflexion models: Bridging the gap between source and high-level models, in: 3rd ACM SIGSOFT Symposium on Foundations of Software

Engineering, pp. 18–28.

[27] Murphy, G.C., Notkin, D., Sullivan, K.J., 2001. Software reflexion models: Bridging the gap between design and implementation. IEEE Transactions on Software Engineering 27, 364–380. doi:10.1109/32.917525.

[28] Nair, V., Hinton, G.E., 2010. Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th international conference on machine learning (ICML-10), pp. 807–814.

[29] Olsson, T., Ericsson, M., Wingkvist, A., 2022. To automatically map source code entities to architectural modules with naive bayes. Journal of Systems and Software 183, 111095. doi:10.1016/j.jss.2021.111095.

[30] Pan, R., Mao, B., Ma, T., Ling, Z., 2026. ArchAgent: Scalable legacy software architecture recovery with LLMs, in: 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 19357–19361. doi:10.1109/ICASSP55912.2026.11462552, arXiv:2601.13007.

[31] Romano, J., Kromrey, J.D., Coraggio, J., Skowronek, J., 2006. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys, in: annual meeting of the Florida Association of Institutional Research. URL: https://web.archive.org/web/20070716131907id_/http://www.florida-air.org/romano06.pdf.

[32] Rukmono, S.A., Ochoa, L., Chaudron, M., 2024. Deductive software architecture recovery via chain-of-thought prompting, in: 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp. 92–96. doi:10.1145/3639476.3639776.

[33] Sinkala, Z.T., Herold, S., 2021. Inmap: Automated interactive code-to-architecture mapping recommendations, in: 18th International Conference on Software Architecture, pp. 173–183.

[34] Stavropoulou, I., Grigoriou, M., Kontogiannis, K., 2017. Case study on which relations to use for clustering-based software architecture recovery. Empirical Software Engineering 22, 1717–1762. doi:10.1007/s10664-016-9459-z.

[35] Tzerpos, V., Holt, R.C., 1997. The orphan adoption problem in architecture maintenance, in: Working Conference on Reverse Engineering, IEEE. pp. 76–82.

[36] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=rJXMpikCZ.

[37] Wilcoxon, F., 1992. Individual comparisons by ranking methods, in: Breakthroughs in statistics: Methodology and distribution. Springer, pp. 196–202.

[38] Wu, W., Hu, H., Fan, Z., Qiao, Y., Huang, Y., Li, Y., Zheng, Z., Lyu, M., 2025. An empirical study of code clones from commercial AI code generators. Proc. ACM Softw. Eng. 2, 2874–2896. doi:10.1145/3729397.

[39] Zhang, G., Jin, W., 2018. Depends: A fast, comprehensive code dependency analysis tool. https://github.com/multilang-depends/depends. Software repository, accessed: 2026-08-28.

[40] Zhang, Y., Xu, Z., Liu, C., Chen, H., Sun, J., Qiu, D., Liu, Y., 2023. Software architecture recovery with information fusion, in: Proceedings of the 31st ACM Joint ESEC/FSE, pp. 1535–1547. doi:10.1145/3611643.3616285.

[41] Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., Koutra, D., 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. Advances in neural information processing systems 33, 7793–7804.